



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

گزارش پیشرفت پروژه اینترنت اشیا - هفته اول

IOT Project Progress Report - Week 1

اعضای گروه

علی فتحی

آرمین فارسی

امیررضا بهرامی

نام درس

اینترنت اشیا

تابستان ۱۴۰۵

نام استاد درس

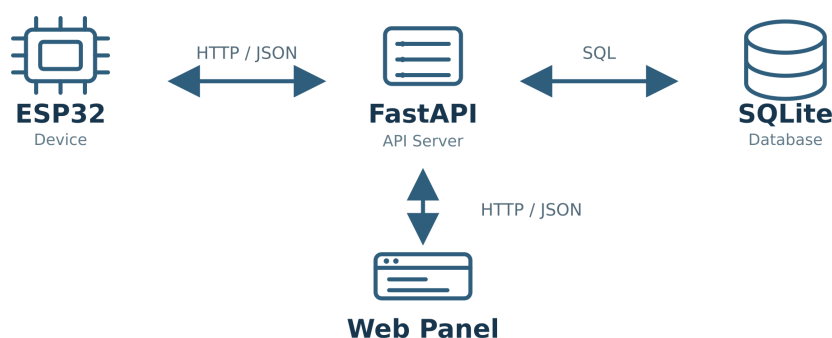
دکتر اجلالی

طراحی و پیاده‌سازی سامانه مدیریت OTA برای ESP32

در هفته اول، قرارداد ارتباطی اجزای سامانه طراحی شد، بورد ESP32 و سرور راه‌اندازی شدند و نسخه اولیه پنل مدیریتی شکل گرفت. نتیجه این مرحله، یک مسیر ارتباطی اولیه و قابل آزمایش از بورد تا سرور، پایگاه داده و پنل بود. دریافت و نصب واقعی نسخه جدید، بررسی مقدار SHA-256 در سمت بورد و مکانیزم Rollback هنوز انجام نشده‌اند و در هفته‌های بعد تکمیل خواهند شد.

۱. هماهنگ‌سازی API‌ها و طراحی Schema پایگاه داده

ابتدا معماری سامانه و قرارداد ارتباط میان سه بخش اصلی مشخص شد. بورد ESP32 از طریق HTTP با سرور ارتباط برقرار می‌کند و پنل مدیریتی نیز از API‌های همان سرور استفاده می‌کند. سرور با FastAPI پیاده‌سازی شده و اطلاعات را در پایگاه داده SQLite نگهداری می‌کند.



شکل ۱: معماری کلی سامانه شامل بورد ESP32، سرور FastAPI، پایگاه داده SQLite و پنل مدیریتی

۱-۱. قرارداد API بین بورد و سرور

دو مسیر اصلی موردنیاز در هفته اول به صورت زیر تعیین شدند:

روش	مسیر	کاربرد
POST	/api/devices/register	ثبت یا به‌روزرسانی اطلاعات دستگاه
GET	/api/devices/{uid}/update	بررسی وجود عملیات به‌روزرسانی برای دستگاه

در درخواست دوم، نسخه فعلی دستگاه با پارامتر **version** در رشته پرس و جو ارسال می شود.

قالب اطلاعات ارسالی هنگام ثبت دستگاه به صورت زیر است:

```
1 {  
2   "device_uid": "ESP32-AABBCCDDEEFF",  
3   "current_version": "1.0.0"  
4 }
```

شناسه دستگاه از آدرس MAC خود بورد ساخته می شود و نسخه فعلی ثابت افزار نیز از اطلاعات برنامه در ESP-IDF خوانده می شود. بنابراین هر دستگاه دارای شناسه پایدار بوده و نسخه ثابت افزار آن نیز به صورت مستقل برای سرور ارسال می شود.

۱-۲. طراحی پایگاه داده

پایگاه داده از چهار جدول اصلی تشکیل شده است.

جدول devices

این جدول اطلاعات دستگاه های ثبت شده، نسخه فعلی، حالت به روزرسانی و زمان آخرین ارتباط آن ها را نگهداری می کند.

فیلد	نوع و محدودیت	کاربرد
id	INTEGER PRIMARY KEY AUTOINCREMENT	شناسه داخلی دستگاه
device_uid	TEXT UNIQUE NOT NULL	شناسه یکتای ساخته شده از آدرس MAC
current_version	TEXT NOT NULL	نسخه فعلی ثابت افزار دستگاه
update_mode	TEXT NOT NULL DEFAULT 'manual'	حالت به روزرسانی؛ یکی از دو مقدار manual یا automatic
last_update_status	TEXT DEFAULT 'idle'	آخرین وضعیت ثبت شده برای به روزرسانی
last_seen_at	DATETIME	زمان آخرین ارتباط دستگاه با سرور
created_at	DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP	زمان ایجاد رکورد
updated_at	DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP	زمان آخرین تغییر رکورد

جدول firmwares

این جدول مشخصات نسخه‌های بارگذاری شده ثابت‌افزار و اطلاعات لازم برای بررسی فایل را نگهداری می‌کند.

فیلد	نوع و محدودیت	کاربرد
id	INTEGER PRIMARY KEY AUTOINCREMENT	شناسه داخلی نسخه
version	TEXT UNIQUE NOT NULL	شماره نسخه ثابت‌افزار
file_name	TEXT NOT NULL	نام فایل بارگذاری شده
file_path	TEXT UNIQUE NOT NULL	مسیر ذخیره‌سازی فایل روی سرور
file_size	INTEGER NOT NULL	اندازه فایل بر حسب بایت
sha256	TEXT NOT NULL	مقدار SHA-256 با طول ۶۴ نویسه
changelog	TEXT	توضیحات تغییرات نسخه
is_active	BOOLEAN NOT NULL DEFAULT 1	فعال یا غیرفعال بودن نسخه
created_at	DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP	زمان ثبت نسخه

جدول update_jobs

هر درخواست به‌روزرسانی برای یک دستگاه به صورت یک رکورد در این جدول ثبت می‌شود.

فیلد	نوع و محدودیت	کاربرد
id	INTEGER PRIMARY KEY AUTOINCREMENT	شناسه عملیات
device_id	INTEGER NOT NULL	ارجاع به دستگاه مقصد
target_firmware_id	INTEGER NOT NULL	ارجاع به نسخه ثابت‌افزار مقصد
from_version	TEXT NOT NULL	نسخه دستگاه پیش از شروع عملیات
trigger_type	TEXT NOT NULL DEFAULT 'manual'	نوع شروع عملیات؛ manual یا automatic
status	TEXT NOT NULL DEFAULT 'pending'	وضعیت جاری عملیات
requested_at	DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP	زمان ایجاد درخواست
started_at	DATETIME	زمان آغاز عملیات
finished_at	DATETIME	زمان پایان عملیات
error_message	TEXT	پیام خطا در صورت شکست عملیات

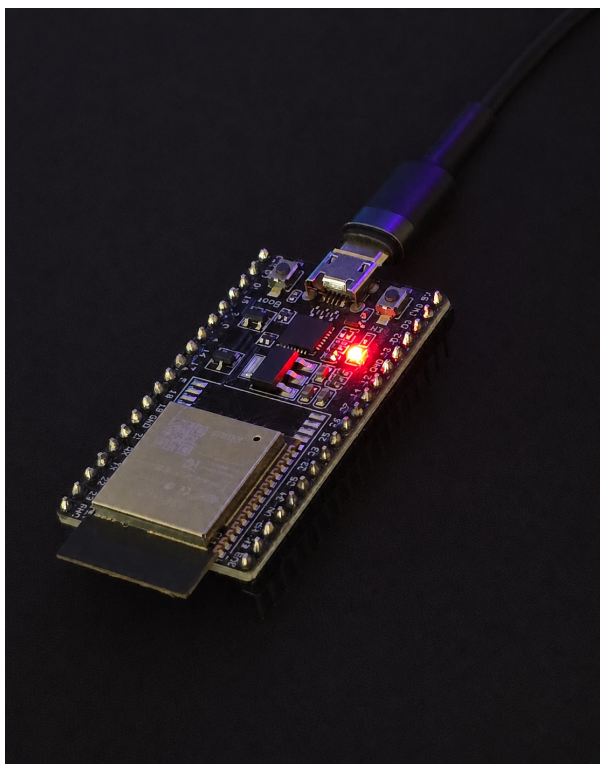
جدول update_events

این جدول رویدادهای مرحله به مرحله هر عملیات را برای ساخت تاریخچه نگهداری می کند.

فیلد	نوع و محدودیت	کاربرد
id	INTEGER PRIMARY KEY AUTOINCREMENT	شناسه رویداد
update_id	INTEGER NOT NULL	ارجاع به عملیات به روزرسانی
status	TEXT NOT NULL	وضعیت گزارش شده
message	TEXT	توضیح تکمیلی رویداد
created_at	DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP	زمان ثبت رویداد

۲. راه اندازی بورد ESP32

در هفته اول بورد ESP32 DevKitC V4 با استفاده از محیط ESP-IDF راه اندازی شد. پس از اطمینان از شناسایی رابط USB-to-UART و امکان Build و Flash برنامه، پروژه اصلی از ابتدا ایجاد شد. برای ساده سازی فرآیند توسعه، چهار اسکریپت برای آماده سازی محیط، ساخت و انتقال برنامه، مشاهده خروجی بورد و پاک سازی فایل های ساخت ایجاد شدند.



شکل ۲: بورد ESP32 DevKitC V4 مورد استفاده در پروژه

۲-۱. اسکریپت آماده‌سازی محیط

در فایل `01_env.sh` مسیر نصب ESP-IDF، پورت سریال و نوع برد مشخص شده و محیط توسعه فعال می‌شود.

```
1 #!/usr/bin/env bash
2
3 export IDF_DIR="$HOME/.espressif/v6.0.2/esp-idf"
4 export ESP_PORT="${ESP_PORT:-/dev/ttyUSB0}"
5 export ESP_TARGET="esp32"
6
7 if [ ! -f "$IDF_DIR/export.sh" ]; then
8     echo "ESP-IDF not found: $IDF_DIR"
9     return 1 2>/dev/null || exit 1
10 fi
11
12 source "$IDF_DIR/export.sh"
```

۲-۲. ساخت و انتقال برنامه روی برد

فایل `02_deploy.sh` ابتدا پروژه را می‌سازد و سپس خروجی را از طریق پورت سریال روی برد منتقل می‌کند. تنظیم نوع برد تنها در اولین اجرای پروژه انجام می‌شود.

```
1 #!/usr/bin/env bash
2
3 set -e
4
5 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
6 PROJECT_ROOT="$(dirname "$SCRIPT_DIR")"
7
8 source "$SCRIPT_DIR/01_env.sh"
9
10 cd "$PROJECT_ROOT"
11
12 if [ ! -f "sdkconfig" ]; then
13     echo "Configuring target: $ESP_TARGET"
14     idf.py set-target "$ESP_TARGET"
15 fi
16
17 echo "Building..."
18 idf.py build
19
20 echo "Flashing to $ESP_PORT..."
21 idf.py -p "$ESP_PORT" flash
22
23 echo "Done."
```

۲-۳. مشاهده خروجی برنامه و پاک‌سازی محیط

برای مشاهده خروجی سریال بورد از `03_run.sh` و برای حذف خروجی‌های ساخت از `04_clean.sh` استفاده می‌شود.

محتوای فایل `03_run.sh`:

```
1 #!/usr/bin/env bash
2
3 set -e
4
5 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
6 PROJECT_ROOT="$(dirname "$SCRIPT_DIR")"
7
8 source "$SCRIPT_DIR/01_env.sh"
9
10 cd "$PROJECT_ROOT"
11
12 idf.py -p "$ESP_PORT" monitor
```

محتوای فایل `04_clean.sh`:

```
1 #!/usr/bin/env bash
2
3 set -e
4
5 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
6 PROJECT_ROOT="$(dirname "$SCRIPT_DIR")"
7
8 source "$SCRIPT_DIR/01_env.sh"
9
10 cd "$PROJECT_ROOT"
11
12 idf.py fullclean
13
14 echo "Build files cleaned."
```

۲-۴. تعریف شناسه دستگاه و نسخه ثابت‌افزار

نسخه پایه ثابت‌افزار در هفته اول برابر 1.0.0 در نظر گرفته شده است. مقدار نسخه در فایل اصلی CMake قرار دارد:

```
1 cmake_minimum_required(VERSION 3.16)
2
3 include($ENV{IDF_PATH}/tools/cmake/project.cmake)
4
5 set(PROJECT_VER "1.0.0")
6
7 project(esp32_ota_client)
```

در زمان اجرا نیز آدرس MAC مورد خوانده شده و از آن یک شناسه یکتا برای دستگاه ساخته می‌شود:

```
1 uint8_t mac[6];
2 ESP_ERROR_CHECK(esp_efuse_mac_get_default(mac));
3
4 char device_id[DEVICE_ID_SIZE];
5 snprintf(
6     device_id,
7     sizeof(device_id),
8     "ESP32-%02X%02X%02X%02X%02X%02X",
9     mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]
10 );
11
12 const esp_app_desc_t *app_desc = esp_app_get_description();
13 const char *firmware_version = app_desc->version;
```

۵-۲. اتصال مورد به Wi-Fi

مورد در حالت Station به شبکه متصل می‌شود. در این بخش رویدادهای مربوط به اتصال و دریافت آدرس IP مدیریت شده و در صورت قطع اتصال، چند بار تلاش مجدد انجام می‌شود. بخش اصلی تابع اتصال به صورت زیر است:

```
1 esp_err_t wifi_connect(void)
2 {
3     wifi_event_group = xEventGroupCreate();
4
5     ESP_ERROR_CHECK(esp_netif_init());
6     ESP_ERROR_CHECK(esp_event_loop_create_default());
7
8     esp_netif_create_default_wifi_sta();
9
10    wifi_init_config_t wifi_init_config =
11        WIFI_INIT_CONFIG_DEFAULT();
12
13    ESP_ERROR_CHECK(
14        esp_wifi_init(&wifi_init_config)
15    );
16
17    ESP_ERROR_CHECK(
18        esp_event_handler_register(
19            WIFI_EVENT,
20            ESP_EVENT_ANY_ID,
21            wifi_event_handler,
22            NULL
23        )
24    );
25
26    ESP_ERROR_CHECK(
27        esp_event_handler_register(
28            IP_EVENT,
29            IP_EVENT_STA_GOT_IP,
30            wifi_event_handler,
```



```

31         NULL
32     )
33 );
34
35 wifi_config_t wifi_config = {
36     .sta = {
37         .ssid = WIFI_SSID,
38         .password = WIFI_PASSWORD,
39     },
40 };
41
42 ESP_ERROR_CHECK(
43     esp_wifi_set_mode(WIFI_MODE_STA)
44 );
45
46 ESP_ERROR_CHECK(
47     esp_wifi_set_config(
48         WIFI_IF_STA,
49         &wifi_config
50     )
51 );
52
53 ESP_ERROR_CHECK(
54     esp_wifi_start()
55 );
56
57 ESP_LOGI(TAG, "Connecting to %s...", WIFI_SSID);
58
59 EventBits_t bits = xEventGroupWaitBits(
60     wifi_event_group,
61     WIFI_CONNECTED_BIT | WIFI_FAILED_BIT,
62     pdFALSE,
63     pdFALSE,
64     portMAX_DELAY
65 );
66
67 if (bits & WIFI_CONNECTED_BIT)
68 {
69     return ESP_OK;
70 }
71
72 return ESP_FAIL;
73 }

```

```

I (3206) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (3236) wifi:dp: 2, bi: 102400, li: 4, scale listen interval from 307200 us to 409600 us
I (3236) wifi:AP's beacon interval = 102400 us, DTIM period = 2
I (3246) wifi:<ba-add>idx:0 (ifx:0, 86:2f:3a:f0:04:0d), tid:0, ssn:1, winSize:64
I (4246) esp_netif_handlers: sta ip: 192.168.128.159, mask: 255.255.255.0, gw: 192.168.128.244
I (4246) wifi: Connected. IP: 192.168.128.159
I (4246) main: Wi-Fi ready.

```

شکل ۳: نمونه خروجی Serial Monitor پس از خواندن شناسه و نسخه ثابت افزار و اتصال موفق به Wi-Fi

۲-۶. آماده‌سازی ارتباط با سرور

پس از اتصال به شبکه، بورد ابتدا خود را در سرور ثبت کرده و سپس وجود عملیات به‌روزرسانی را بررسی می‌کند. در هفته اول تنها درخواست و دریافت پاسخ انجام می‌شود و دریافت یا نصب فایل ثابت‌افزار هنوز آغاز نمی‌شود.

```
1 ESP_LOGI(TAG, "Registering device on server...");
2 if (api_register_device(device_id, firmware_version) != ESP_OK)
3 {
4     ESP_LOGE(TAG, "Device registration failed.");
5     return;
6 }
7
8 ESP_LOGI(TAG, "Device registered successfully.");
9
10 ESP_LOGI(TAG, "Checking for update...");
11 if (api_check_for_update(device_id, firmware_version) != ESP_OK)
12 {
13     ESP_LOGE(TAG, "Update check failed.");
14     return;
15 }
16
17 ESP_LOGI(TAG, "Week-1 ESP32 flow completed successfully.");
```

۳. راه‌اندازی سرور و پنل مدیریتی

سرور با زبان Python و چارچوب FastAPI پیاده‌سازی شده است. هنگام شروع برنامه، پایگاه داده SQLite به صورت خودکار آماده می‌شود و فایل‌های پنل مدیریتی نیز از همان سرور در دسترس قرار می‌گیرند. برای آماده‌سازی و اجرای این بخش، دو اسکریپت مجزا در نظر گرفته شده است.

۳-۱. راه‌اندازی سرور

اسکریپت 01_setup.sh یک محیط مجازی Python ایجاد کرده و وابستگی‌های پروژه را نصب می‌کند. سپس با 02_run.sh سرور روی تمام رابط‌های شبکه و پورت 8000 اجرا می‌شود.

محتوای فایل 01_setup.sh:

```
1 #!/usr/bin/env bash
2
3 set -e
4
5 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
6 PROJECT_ROOT="$(dirname "$SCRIPT_DIR")"
7
8 cd "$PROJECT_ROOT"
9
10 if [ ! -d ".venv" ]; then
11     echo "Creating virtualenv..."
```

```

12     python3 -m venv .venv
13 fi
14
15 ./venv/bin/pip install --upgrade pip
16 ./venv/bin/pip install -r requirements.txt
17
18 echo "Setup done."

```

محتوای فایل 02_run.sh:

```

1 #!/usr/bin/env bash
2
3 set -e
4
5 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
6 PROJECT_ROOT="$(dirname "$SCRIPT_DIR")"
7
8 HOST="${OTA_HOST:-0.0.0.0}"
9 PORT="${OTA_PORT:-8000}"
10
11 cd "$PROJECT_ROOT"
12
13 if [ ! -d ".venv" ]; then
14     echo "Run scripts/01_setup.sh first."
15     exit 1
16 fi
17
18 echo "Panel: http://localhost:$PORT/panel/   Docs: http://localhost:$PORT/docs"
19 exec ./venv/bin/uvicorn app.main:app --host "$HOST" --port "$PORT" --reload

```

در سرور یک مسیر ساده برای بررسی سلامت سرویس نیز وجود دارد:

```

1 @app.get("/api/health")
2 def health():
3     return {"status": "ok"}

```

```

ali@ali-System-Product-Name:~/Desktop/SUT/IOT/src/server$ ./scripts/02_run.sh
Panel: http://localhost:8000/panel/ Docs: http://localhost:8000/docs
INFO: Will watch for changes in these directories: ['/home/ali/Desktop/SUT/IOT/src/server']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [273094] using WatchFiles
INFO: Started server process [273099]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 192.168.128.159:60862 - "POST /api/devices/register HTTP/1.1" 200 OK
INFO: 192.168.128.159:60863 - "GET /api/devices/ESP32-1CC3ABB3AAB8/update?version=1.0.0 HTTP/1.1" 200 OK

```

```

I (4266) api_client: POST http://192.168.128.216:8000/api/devices/register
I (4266) api_client: Body: {"device_uid":"ESP32-1CC3ABB3AAB8","current_version":"1.0.0"}
I (9076) api_client: Response status: 200
I (9076) api_client: Response body: {"id":1,"device_uid":"ESP32-1CC3ABB3AAB8","current_version":"1.0.0","update_mode":"manual","registered":false}
I (9086) main: Device registered successfully.
I (9086) main: Checking for update...
I (9096) api_client: GET http://192.168.128.216:8000/api/devices/ESP32-1CC3ABB3AAB8/update?version=1.0.0
I (9136) api_client: Response status: 200
I (9136) api_client: Response body: {"update_available":false,"current_version":"1.0.0"}
I (9136) main: Week-1 ESP32 flow completed successfully.
I (9136) main_task: Returned from app_main()

```

شکل ۴: اجرای سرور FastAPI و دریافت درخواست‌های ارسالی از بورد ESP32

۲-۳. مدیریت دستگاه‌ها

مسیر ثبت دستگاه به گونه‌ای پیاده‌سازی شده که اگر شناسه برای اولین بار مشاهده شود، یک رکورد جدید ایجاد کند و در دفعات بعد نسخه فعلی و زمان آخرین ارتباط را به‌روزرسانی نماید. این رفتار باعث می‌شود همان درخواست ثبت دستگاه بتواند در ادامه پروژه به عنوان پیام دوره‌ای حضور دستگاه نیز استفاده شود.

```

1 @router.post("/register")
2 def register_device(body: RegisterIn, conn: sqlite3.Connection = Depends(get_db)):
3     """Called by the board on boot and as a periodic heartbeat."""
4     existing = conn.execute(
5         "SELECT id FROM devices WHERE device_uid = ?", (body.device_uid,)
6     ).fetchone()
7
8     if existing is None:
9         conn.execute(
10             """INSERT INTO devices (device_uid, current_version, update_mode,
11 last_seen_at)
12 VALUES (?, ?, ?, CURRENT_TIMESTAMP)""",
13             (body.device_uid, body.current_version, body.update_mode or "manual"),
14         )
15     else:
16         # Mode is owned by the panel, so only overwrite it if the board sent one.
17         conn.execute(
18             """UPDATE devices
19 SET current_version = ?,
20 update_mode = COALESCE(?, update_mode),
21 last_seen_at = CURRENT_TIMESTAMP,
22 updated_at = CURRENT_TIMESTAMP
23 WHERE id = ?""",
24             (body.current_version, body.update_mode, existing["id"]),

```

```

24     )
25
26     device = get_device_or_404(conn, body.device_uid)
27     return {
28         "id": device["id"],
29         "device_uid": device["device_uid"],
30         "current_version": device["current_version"],
31         "update_mode": device["update_mode"],
32         "registered": existing is None,
33     }

```

۳-۳. پنل مدیریتی

پنل مدیریتی با HTML، CSS و JavaScript پیاده‌سازی شده و چهار صفحه برای دستگاه‌ها، نسخه‌های ثابت‌افزار، عملیات‌های به‌روزرسانی و تاریخچه دارد. در هفته اول، نمایش دستگاه‌های ثبت‌شده، نسخه فعلی، حالت به‌روزرسانی و زمان آخرین ارتباط آماده شد. پنل همچنین با مسیر `/api/health` در دسترس بودن سرور را بررسی می‌کند.

شکل ۵: صفحه دستگاه‌ها در پنل مدیریتی و نمایش شناسه، نسخه فعلی و وضعیت دستگاه

شکل ۶: صفحه مدیریت نسخه‌های ثابت‌افزار و فرم بارگذاری فایل BIN

۴. آزمایش زنجیره هفته اول

برای آزمایش ارتباط بخش‌ها، سرور روی رایانه اجرا شد و بورد و رایانه به یک شبکه مشترک متصل شدند. آدرس شبکه رایانه در مقدار `SERVER_BASE_URL` قرار گرفت. بورد پس از راه‌اندازی، شناسه و نسخه خود را استخراج کرد، به Wi-Fi متصل شد و درخواست ثبت را به سرور فرستاد. سپس درخواست بررسی نسخه جدید نیز ارسال شد. در ادامه، رکورد ایجادشده در جدول `devices` بررسی شد تا ثبت شناسه، نسخه فعلی، حالت به‌روزرسانی و زمان آخرین ارتباط تأیید شود.

```
ali@ali-System-Product-Name:~/Desktop/SUT/IOT/src/server/storage$ sqlite3 ota.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .headers on
sqlite> .mode column
sqlite> SELECT * FROM devices;
id device_uid current_version update_mode last_update_status last_seen_at created_at updated_at
-----
1 ESP32-1CC3ABB3AAB8 1.0.0 manual idle 2026-08-18 07:38:17 2026-08-17 21:35:16 2026-08-18 07:38:17
```

شکل ۷: رکورد دستگاه ثبت‌شده در پایگاه داده SQLite

با تکمیل این مسیر، زیرساخت لازم برای شروع پیاده‌سازی واقعی OTA آماده شد. ثابت‌افزار نسخه 1.0.0 در وضعیت فعلی شناسه دستگاه و نسخه برنامه را می‌خواند، به شبکه متصل می‌شود، دستگاه را در سرور ثبت می‌کند و وجود به‌روزرسانی را بررسی می‌کند.

۵. جمع‌بندی هفته اول

خروجی‌های اصلی هفته اول عبارت‌اند از:

- طراحی قرارداد API بین بورد، سرور و پنل مدیریتی؛
- طراحی کامل ساختار اولیه پایگاه داده SQLite؛
- راه‌اندازی بورد و آماده‌سازی فرآیند Build, Flash, Monitor و پاک‌سازی محیط؛
- تعریف شناسه یکتای دستگاه و نسخه اولیه ثابت‌افزار 1.0.0؛
- اتصال بورد به Wi-Fi و آماده‌سازی ارتباط HTTP با سرور؛
- راه‌اندازی سرور FastAPI و پنل مدیریتی؛
- آماده‌سازی مسیر سراسری ثبت دستگاه و نمایش اطلاعات آن در سامانه.

در سمت سرور، محاسبه مقدار SHA-256 هنگام بارگذاری فایل آماده شده است؛ با این حال، دریافت و نصب واقعی ثابت‌افزار، بررسی این مقدار در سمت بورد، تنظیم پارتیشن‌های OTA و مکانیزم Rollback هنوز پیاده‌سازی نشده‌اند و در هفته‌های بعد تکمیل خواهند شد. افزودن یک رفتار قابل مشاهده مانند چشمک‌زدن LED برای نمایش تفاوت نسخه‌های ثابت‌افزار نیز به دلیل در دسترس نبودن قطعات جانبی به مرحله بعد موکول شد.